Introduction

- **Machine language** is the low level programming language, **that *can only* be represented by *0s and 1s*.**
- **I**n earlier when we have to create a picture or show data on the screen of the computer then it is very difficult to draw using only binary digits(0s and 1s).
- **For example**: To write 120 in the computer system its representation is 1111000. So it is very difficult to learn.
- **To overcome** this problem **the assembly language** is invented.

Assembly language

- **Assembly language is the more than low level and less than high-level language so it is intermediary language**.
- Assembly languages use numbers, symbols, and abbreviations instead of 0s and 1s.
- **For example**: For addition, subtraction and multiplications it uses symbols likes Add, sub and Mul, etc.



Data types

In assembly language, data types and definitions **establish the size and type of data used by a program**, **while** instructions dictate how the processor should move, manipulate, and control the flow of that data. Since assembly language works directly with the machine's architecture, data types correspond to the size of data the hardware can process.

Standard data types for the x86 architecture, include:

Byte: An 8-bit integer. Many assemblers use the DB or BYTE directive to define a byte.

Word: A 16-bit integer, defined with DW or WORD.

Doubleword (Dword): A 32-bit integer, defined with DD or DWORD.

Quadword (Qword): A 64-bit integer, defined with DQ or QWORD.

Floating-point: Data types for representing real numbers, such as single-precision (32-bit) and double-precision (64-bit).

8086 assembly instructions categorized based on their functionality

- **Data Transfer Instructions**

1. `MOV`: Move data from one location to another.
2. `PUSH`: Push data onto the stack.
3. `POP`: Pop data from the stack.
4. `XCHG`: Exchange the values of two operands.
5. `IN`: Input data from a port.
6. `OUT`: Output data to a port.
7. `LEA`: Load effective address.
8. `LDS`: Load pointer using DS.
9. `LES`: Load pointer using ES.
10. `LDT`: Load task register.
11. `MOVSX`: Move with sign extension.
12. `MOVZX`: Move with zero extension.

8086 assembly instructions categorized based on their functionality

- **Arithmetic Instructions**

1. `ADD`: Add two operands.
2. `SUB`: Subtract one operand from another.
3. `INC`: Increment a value by one.
4. `DEC`: Decrement a value by one.
5. `MUL`: Unsigned multiply.
6. `IMUL`: Signed multiply.
7. `DIV`: Unsigned divide.
8. `IDIV`: Signed divide.
9. `ADC`: Add with carry.
10. `SBB`: Subtract with borrow.
11. `DAA`: Decimal adjust for addition.
12. `DAS`: Decimal adjust for subtraction.
13. `AAM`: ASCII adjust AX before multiplication.
14. `AAD`: ASCII adjust AX before division.

8086 assembly instructions categorized based on their functionality

Logical Instructions

1. AND: Logical AND.
2. OR: Logical OR.
3. XOR: Logical XOR.
4. NOT: Logical NOT.
5. SHL: Shift left.
6. SHR: Shift right.
7. SAR: Shift right arithmetic.
8. ROL: Rotate left.
9. ROR: Rotate right.
10. RCL: Rotate through carry left.
11. RCR: Rotate through carry right.

8086 assembly instructions categorized based on their functionality

- **Control Flow Instructions**

1. `JMP`: Unconditional jump.
2. `CALL`: Call a procedure.
3. `RET`: Return from a procedure.
4. `JE`: Jump if equal.
5. `JNE`: Jump if not equal.
6. `JG`: Jump if greater.
7. `JL`: Jump if less.
8. `JGE`: Jump if greater or equal.
9. `JLE`: Jump if less or equal.
10. `JC`: Jump if carry.
11. `JNC`: Jump if not carry.
12. `JO`: Jump if overflow.
13. `JNO`: Jump if not overflow.
14. `JS`: Jump if sign.
15. `JNS`: Jump if not sign.

8086 assembly instructions categorized based on their functionality

Interrupt Instructions

1. `INT`: Interrupt.
2. `IRET`: Interrupt return.
3. `INT3`: Breakpoint interrupt.
4. `INTO`: Interrupt on overflow.

Comparison Instructions

1. `CMP`: Compare two operands.
2. `TEST`: Logical compare (AND) two operands.

8086 assembly instructions categorized based on their functionality

Flag Control Instructions

1. `CLC`: Clear carry flag.
2. `STC`: Set carry flag.
3. `CLI`: Clear interrupt flag.
4. `STI`: Set interrupt flag.
5. `CLD`: Clear direction flag.
6. `STD`: Set direction flag.